

1. Prove that the language of even-length non-palindromes is a context-free language

To prove that even length non-palindromes is a context-free language we can come with a grammar that recognizes such language as follows:

$$S \rightarrow aS1a \mid bS1b \mid S1$$

$$S1 \rightarrow aAb \mid bAa$$

$$A \rightarrow aAa \mid bAb \mid \varepsilon$$

The grammar above will generate only even-length strings that are not palindrome. Therefore, since we found a grammar that generates even-length non-palindrome strings, then we can conclude that such language is a context free language.

1. Use the pumping lemma for context-free languages to prove that the following language is not context-free:

$$L = \{ww \mid w \in \Sigma^*\}$$

Let $\Sigma^* = \{0,1\}^*$, we can proceed with the pumping lemma proof.

Assume, for a contradiction, that L is context free. Then L has a length $m > p$, where p is the pumping length. Now, to find a string that fits the pumping requirements let s , a string in the language, be equal to $0^p 1^p 0^p 1^p$.

$$s = 0^p 1^p 0^p 1^p$$

According to the pumping lemma $|s| \geq p$, where;

$$|vy| > 0$$

$$|vxy| \leq p$$

$$s = uv^i xy^i z, \text{ a string in the language for all } i \geq 0.$$

Case 1: choose $i = 0$.

From the condition above, we can tell that vxy contains all 0's within either the first or second string of 0's. Now we have $s = uv^0 xy^0 z = 0^{p-k} 1^p 0^p 1^p$ or $0^p 1^p 0^{p-k} 1^p$, where k is the number of 0's contained in vy , and since $|vy| > 0$ either v or y must contain at least a 0. This forces either the first or second string of 0's to have at least one fewer 0's than the other. Thus, $s \notin L$ and this is a contradiction of the pumping lemma.

Case 2: choose $i=0$.

From the condition above we can tell that vxy contains all 1's within either the first or second string of 1's. By using the same approach as in case 1, we can see that we get a contradiction as well.

Case 3:

vxy is comprised of a mix of 0's and 1's. This really describes two cases, where vxy is a string of 0's followed by a string of 1's or vxy is a string of 1's followed by string of 0's. We take the first case to be representative. In this case vxy either straddles the first 0-1 divisions or it straddles the second 0-1 divisions. Again, because $|vxy| \leq p$, it follows that pumping either up or down will only affect the substrings immediately adjacent to the division that is straddled. The other two substrings will be unaffected. Thus the length of the straddled substrings will be changed by pumping while the length of the other two will not be. Therefore, the result of pumping will result in a string that is not in the language, and this is again a contradiction to the pumping lemma.

In every case, s cannot be pumped, and therefore, we have a contradiction with the pumping lemma. Thus, our original assumption was false and L is not context free.

2. Use the observation that $\{a^n b^n c^n \mid n \geq 0\}$ is not context-free to prove that context-free languages are not closed under intersection.

First, having the language $L1 = \{a^i b^j c^j \mid i, j \geq 0\}$ we can easily generate a grammar to prove that such language is context free, the grammar is as follows:

$S \rightarrow AC \mid \epsilon$
 $A \rightarrow aAb \mid \epsilon$
 $C \rightarrow Cc \mid \epsilon$

Second, having the language $L2 = \{a^j b^i c^i \mid i, j \geq 0\}$ we can easily generate a grammar to prove that such language is context free, the grammar is as follows:

$S \rightarrow AC \mid \epsilon$
 $A \rightarrow Aa \mid \epsilon$
 $C \rightarrow bCc \mid \epsilon$

Finally, after proving that $L1$ and $L2$ are context free, we can prove that the intersection of those two context free languages will result in non-context free language as follows:

$$L = L1 \cap L2 = \{a^n b^n c^n \mid n \geq 0\}$$

As we can see, $L1$ and $L2$ are both context free languages. However, the intersection of both context free languages is equal to a non-context free language. This proves that context free languages are not closed under intersection.

3. Use this result and DeMorgan's laws to prove that context-free languages are not closed under complement.

Using DeMorgan's theorem we can conclude the following:

Taking into account the fact that context free languages are closed under union.

Assume, for a contradiction, that context-free languages are closed under complement. Then, the inverse of the union of two context free languages must give as a result another context free language.

$$\begin{aligned} L1 \cup L1 &= L \rightarrow \text{context free language} \\ \overline{L1 \cup L2} &= \overline{L1} \cap \overline{L2} \\ \overline{\overline{L1} \cup \overline{L2}} &= L1 \cap L2 \end{aligned}$$

As we can see, the union of two inversed context free languages is equal to the inverse of the intersection of both context free languages. Furthermore, the inverse of the union of two inversed context free language, according to our assumption, must be equal to another context free language. Instead, this is equal to the intersection of both context free languages, which we have already proved is not a context free language. Therefore, this is a contradiction and our assumption that context free languages are closed under complement is wrong.

4. If A and B are languages, let $A \triangleleft B$ be $\{xy \mid x \text{ in } A \text{ and } y \text{ in } B, \text{ and } |x| = |y|\}$. Show that if A and B are regular, $A \triangleleft B$ is context-free.

We will construct a PDA, M, that will recognize $A \triangleleft B$. Since A and B are regular there are DFA'S Da and Db such that $L(Da) = A$ and $L(Db) = B$. Informally we construct M as follows. From a new start state we first push \$ onto the stack to mark the stack's bottom and transitions to the start state of Da. Then for every symbol it sees from A it pushes a 0 onto the stack for counting. It non-deterministically guesses the end of the string from A and transition to the start symbol of Db if it is in a final state of Da. It then pops a 0 for every symbol from the string in B and only moves the accepting state if it is currently in a final state of Db and is at the bottom of the stack.

Clearly this will recognize $A \triangleleft B$ since we non-deterministically guess the middle of the string and keep track with our stack to ensure that $|x| = |y|$.

Formally, we define M as follows. We have $Da = (Qa, \Sigma, \delta_a, q_a, Fa)$ and $Db = (Qb, \Sigma, \delta_b, q_b, Fb)$. Define $M = (Q, \Sigma, \delta, q_0, F)$ where:

$Q = Qa \cup Qb \cup \{q_0, q_f\}$, states union from Da and Db, + a new start state and final state.

Σ is the same for all languages

$R = \{\$, 0\}$, \$ to mark the beginning of the stack and 0 to count how many symbols have been seen in the string from A.

$\delta(q, a, b)$	$= \{qa, \$\}$	if $q = q_0, a = b = \varepsilon$
	$= \{\delta(q, a), 0\}$	if q is in Q_a , a is in Σ , $b = \varepsilon$
	$= \{qb, \varepsilon\}$	if q is in F_a , $a = b = \varepsilon$
	$= \{\delta(q, a), \varepsilon\}$	if q is in Q_b , a is in Σ , $b = 0$
	$= \{qf, \varepsilon\}$	if q is in F_b , $a = \varepsilon$, $b = \$$
	$= \emptyset$	otherwise

The transition function works as informally described above. Since we are able to build a PDA for the language $A \triangleleft B$ be $\{xy \mid x \text{ in } A \text{ and } y \text{ in } B, \text{ and } |x| = |y|\}$, we have shown that $A \triangleleft B$ is context-free.